

Tarea C — Recuperación léxica contra semántica latente sobre un corpus mínimo

Objetivo

Comparar un recuperador léxico (TF-IDF con `TfidfVectorizer` de scikit-learn, parámetros por defecto, y similitud coseno) contra una proyección semántica latente (LSA mediante `TruncatedSVD` a 4 componentes) sobre un corpus de diez documentos (d01–d10) y seis consultas (q1–q6) con un documento relevante cada una (tarea-c-recuperacion-lexica-lsa.pdf). Se evalúa con Hit@1, Hit@3 y MRR, se identifican las consultas donde cada método falla y se discute por qué un recuperador denso resolvería mejor el caso fallido, así como su costo frente a TF-IDF.

Método

Cada consulta se representa con el mismo vectorizador TF-IDF ajustado sobre el corpus y se ordenan los diez documentos por similitud coseno descendente, la medida estándar para comparar la orientación de vectores de texto (s2-rag-y-vector-search.md). Hit@k es la proporción de consultas cuyo relevante queda en el top-k y MRR la media de 1/rank del relevante (s2-rag-y-vector-search.md). En la Parte 2, la matriz TF-IDF se proyecta con `TruncatedSVD(n_components=4, random_state=0)` ajustado sobre el corpus; documentos y consultas se normalizan L2 y se repite el protocolo. Se define *fallo* como relevante fuera del top 3. Los scripts son autocontenidos (sin descargas); código condensado (textos completos en el enunciado):

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import normalize

# Corpus d01-d10 y consultas q1-q6 con los textos exactos del enunciado
# y juicios: q1->d04, q2->d03, q3->d08, q4->d05, q5->d09, q6->d02.
corpus = {"d01": "El mecanismo de atención pondera cada token ...",
          ..., "d10": "La cuantización reduce la precisión de los pesos a 8 o 4 bits .."}
consultas = {"q1": ("¿Qué función de ranking léxica pondera la frecuencia de términos?", "
                  ..., "q6": ("¿Qué arquitectura combina autoatención con capas feed-forward?", "

ids = list(corpus)
vec = TfidfVectorizer()                    # parámetros por defecto
X = vec.fit_transform(corpus.values())    # matriz TF-IDF del corpus
Q = vec.transform([t for t, _ in consultas.values()])

def evaluar(S):                           # S: similitudes (6 x 10)
    rk = [list(np.array(ids)[np.argsort(-s)]).index(rel) + 1
           for s, (_, rel) in zip(S, consultas.values())]
    n = len(rk)
    return (sum(r == 1 for r in rk)/n,    # Hit@1
            sum(r <= 3 for r in rk)/n,    # Hit@3
            sum(1/r for r in rk)/n)       # MRR

print(evaluar(cosine_similarity(Q, X)))    # Parte 1: TF-IDF + coseno

svd = TruncatedSVD(n_components=4, random_state=0)
X1 = normalize(svd.fit_transform(X))      # Parte 2: LSA
Q1 = normalize(svd.transform(Q))
print(evaluar(cosine_similarity(Q1, X1)))
```

Resultados

Tabla 1 — Comparación global (Parte 2), seis consultas:

Método	Hit@1	Hit@3	MRR
--------	-------	-------	-----

TF-IDF	0.8333	0.8333	0.8611
LSA (SVD 4)	0.6667	0.8333	0.75

Tabla 2 — Análisis por consulta (Parte 3); fallo = relevante fuera del top 3:

Consulta	Relevante	Rank TF-IDF	Rank LSA	Fallo TF-IDF	Fallo LSA	Términos compartidos consulta–relevante (df)
q1	d04	1	1	no	no	8: frecuencia, léxica, ranking, términos (df 1)...
q2	d03	1	1	no	no	3: fragmentos, prompt, al (df 1)
q3	d08	1	1	no	no	5: matrices, rango, entrena, bajo (df 1)...
q4	d05	1	3	no	no	4: orientación, vectores, compara (df 1), la (df 6)
q5	d09	6	6	sí	sí	1: solo «la» (df 6)
q6	d02	1	1	no	no	5: autoatención, capas, feed, forward (df 1)...

TF-IDF coloca el relevante en el puesto 1 en cinco consultas (p. ej., q1: d04 con similitud 0.689076) y falla solo en q5: d09 queda en el puesto 6 con similitud 0.079002, por detrás de d03 (0.310676), d06 (0.200851) y d04 (0.189560). LSA pierde precisión en el puesto 1 (0.6667): en q4 el relevante d05 baja al puesto 3, superado por d01 (+0.9220) y d10 (+0.9206), y en q5 d09 también queda en el puesto 6 (+0.4999). Las cuatro componentes SVD explican 0.3953 de la varianza (0.0342, 0.1266, 0.1160, 0.1186). Ambos métodos fallan únicamente en q5.

Discusión

El fallo común: q5. La consulta q5 («¿Qué hace que el modelo sea más determinista al elegir la siguiente palabra?») y su relevante d09 («La temperatura reescala los logits antes del softmax; valores bajos concentran la probabilidad») comparten un único término, «la» (df = 6, presente en 6 de los 10 documentos): ningún término de contenido. d09 expresa el determinismo con el vocabulario «temperatura», «logits», «softmax» y «probabilidad», mientras la consulta usa «determinista», «elegir», «siguiente» y «palabra»; sin solapamiento léxico, TF-IDF solo puede apoyarse en palabras funcionales o genéricas y d09 cae al puesto 6. En las consultas resueltas, en cambio, consulta y relevante comparten términos raros y discriminativos (df = 1): «frecuencia», «léxica», «ranking», «términos» en q1; «fragmentos», «prompt» en q2; «matrices», «rango», «entrena» en q3; «autoatención», «capas», «feed», «forward» en q6. LSA tampoco salva q5: su espacio latente se aprende de las co-ocurrencias de este corpus de solo 10 documentos, donde «determinista» nunca co-ocurre con «temperatura» o «softmax»; además, con 4 componentes que explican 0.3953 de la varianza, la proyección mezcla temas y en q4 hace que d01 y d10 superen a d05 (aunque sin salir del top 3).

Recuperador denso (Parte 4). Un recuperador denso con embeddings de un modelo entrenado a gran escala resolvería q5 mejor porque captura sinonimia y paráfrasis sin solapamiento de términos: puede acercar «más determinista al elegir la siguiente palabra» y «valores bajos concentran la probabilidad» en el espacio vectorial, exactamente el hueco que deja d09 en el puesto 6 tanto en TF-IDF (0.079002) como en LSA (+0.4999). Las notas del curso plantean esta limitación de los métodos dispersos: TF-IDF/BM25 son exactos con términos técnicos, siglas y nombres propios, pero no entienden sinónimos ni paráfrasis (s2-rag-y-vector-search.md); los embeddings densos cubren justamente ese caso. El costo frente a TF-IDF es triple: (1) requiere un modelo preentrenado y cómputo de inferencia por consulta (GPU/CPU, latencia), frente a TF-IDF, que no necesita entrenamiento y es ligero; (2) exige memoria para almacenar el índice de embeddings de todos los documentos; y (3) es menos interpretable, porque la similitud entre vectores densos no se descompone en términos compartidos como la de TF-IDF (s2-rag-y-vector-search.md). En un corpus mínimo como este, ese costo no se compensa: TF-IDF ya resuelve 5 de 6 consultas en el puesto 1 y el fallo restante es de cobertura de vocabulario, no de ingeniería.